Design, Implementation and Cloud Evaluation of a Fault-Tolerant, Horizontally Sharded Durable Execution Engine

A PROJECT REPORT

Submitted by

BL.EN.U4AID23007 <NAME 1>

BL.EN.U4AID23031 <NAME 2>

BL.EN.U4AID230XX <NAME 3>

BL.EN.U4AID23065 <NAME 4>

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE, BENGALURU

AMRITA VISHWA VIDYAPEETHAM

BENGALURU 560 035

DECEMBER 2026

ACKNOWLEDGEMENTS

The satisfaction that accompanies the successful completion of any task would be incomplete without mention of the people who made it possible, and whose constant encouragement and guidance have been a source of inspiration throughout the course of this project work.

We offer our sincere pranams at the lotus feet of **"AMMA", MATA AMRITANANDAMAYI DEVI**, who showered her blessing upon us throughout the course of this project work.

We owe our gratitude to **Prof. Manoj P.**, Director, Amrita Vishwa Vidyapeetham Bengaluru Campus. We would like to place our heartfelt gratitude to **Dr. Gopalakrishnan E.A.**, Principal, Amrita School of Computing and Amrita School of Artificial Intelligence, Bengaluru, for his valuable support and inspiration.

It is a great pleasure to express our gratitude and indebtedness to our project guide **<GUIDE NAME AND DESIGNATION>**, Department of Computer Science and Engineering, Amrita School of Computing, Bengaluru, for the valuable guidance, encouragement, moral support and affection extended to us throughout the project work.

We express our gratitude to the project panel members for their suggestions, encouragement and moral support during the course of the project, and to all faculty members for their academic support. Finally, we are forever grateful to our parents, who have loved, supported and encouraged us in all our endeavours.

ABSTRACT

A long-running workflow, such as an order, a payment or a multi-step software agent, changes the state of several external services, and a crash part way through it either loses paid-for work or repeats work that must not happen twice. Durable execution engines prevent this by journalling every workflow decision and recovering by deterministic replay, but production engines depend on four to six infrastructure services. This report presents *durexec*, a durable execution engine in Go whose only dependency is PostgreSQL. Workflows are hashed to a fixed set of shards; each shard is owned through a lease held in a database row, and a fencing token re-checked inside every write transaction stops a node that has lost its lease from committing. A journalled semantic check lets the engine recognise a reworded retry of an action already taken by a language-model agent. On Amazon EKS with three engine nodes and one PostgreSQL host, a closed-loop workload reached 131 workflows per second at a 99th-percentile latency of 3.1 s, limited by database CPU, and the sustainable open-loop rate was about 100 workflows per second. Workers, shards and engine replicas changed throughput by at most 19%, and extra replicas gave no gain while the database was the bottleneck. In sixteen fault-injection trials, covering engine termination, a 35 s process freeze, loss of database connections, a node drain and a database reboot, every accepted workflow completed, no journal invariant was violated and every write by a stale owner was rejected.

Keywords: durable execution, event sourcing, deterministic replay, leases, fencing tokens, sharding, fault tolerance, PostgreSQL.

TABLE OF CONTENTS

Page No.

ACKNOWLEDGEMENTS i

ABSTRACT ii

LIST OF FIGURES v

LIST OF TABLES vii

CHAPTER 1 – INTRODUCTION 1

1.1 Background 1

1.2 Motivation 3

1.3 Problem Statement 4

1.4 Objectives 4

1.5 Scope and Limitations. 4

1.6 Contributions 5

1.7 Organisation of the Report 6

CHAPTER 2 – LITERATURE SURVEY 7

2.1 Durable Execution and Stateful Serverless Workflows 7

2.2 Partitioning Work Across Machines 8

2.3 Leases, Coordination and Fencing 9

2.4 Long-Running Transactions and Sagas 10

2.5 Failure Modes of Agent Systems 10

2.6 Semantic Similarity and Caching 11

2.7 Comparison of Related Systems 12

2.8 Research Gap 12

CHAPTER 3 – REQUIREMENTS AND ANALYSIS 14

3.1 Functional Requirements 14

3.2 Non-Functional Requirements 14

3.3 Software and Hardware Requirements 14

3.4 Assumptions and Constraints 14
3.5 Analysis of the Chosen Approach 17
CHAPTER 4 – SYSTEM DESIGN 19
4.1 High-Level Design 19
4.2 Low-Level Design 23
REFERENCES 29

LIST OF FIGURES

4.1 System architecture 20
4.2 Two-level mapping of workflows to shards and nodes 22
4.3 Lease expiry and fencing of a stale owner 23
4.4 The workflow task loop 24

LIST OF TABLES

1.1 Terminology used in this report. 2

1.2 Project objectives 5

2.1 Comparison of related systems 12

3.1 Functional requirements 15

3.2 Non-functional requirements 16

3.3 Hardware used for development and evaluation 16

4.1 Persistent data model 23

4.2 Crash windows for one activity execution 26

CHAPTER - 1

INTRODUCTION

1.1 Background

Much of the software that runs a business is not a single request answered in milliseconds but a process that unfolds over seconds, hours or days. Placing a food order charges a card, reserves stock, notifies a restaurant and assigns a driver. Settling an invoice debits one ledger, credits another and files a tax record. An agent built on a large language model that is asked to arrange travel plans an itinerary, calls a booking service, waits for a person to approve the result and then confirms it. Each of these is a *workflow*: a sequence of steps, spread over several independent services, in which every step depends on the results of the steps before it and most steps change the state of some system outside the workflow itself.

Two properties make workflows hard to run reliably. A workflow is stateful, because step four only makes sense in the light of what steps one to three decided. Its steps also have external side effects that the workflow cannot withdraw: once a payment provider has moved money, no local rollback can move it back. A database transaction has neither difficulty, since it is short enough to hold its locks throughout and its effects are confined to a system that can undo them. A workflow has both.

As a result, an ordinary crash becomes a correctness problem. Suppose the order workflow is interrupted after the card has been charged and the stock reserved, but before the restaurant has been told. A system that has kept no durable record of its progress has two choices when it restarts. It can run the workflow again from the beginning, charging the card a second time, or it can abandon the workflow, keeping the customer's money for an order that will never be delivered. Without a record of what was already done, there is no correct third option.

The usual responses in practice are retry loops, periodic jobs that poll a status column, and message queues whose consumers record progress in application tables. Each works for the workflow it was written for, and each must be written again for the next one.

Durable execution is the name given to systems that solve the problem once for every workflow built on them [1, 2]. The idea on which they rest, and on which this report rests, is that *every decision a workflow makes is appended to a log before it is acted on, and recovery is performed by running the workflow's own code again against that log*. Completed steps return their recorded results instead of executing a second time, so the re-executed code arrives at the point the failed process had reached. Because nothing private to one machine is part of the workflow's state, the recovery can take place on a different machine.

Table 1.1 fixes the terms used throughout the report.

Table 1.1: Terminology used in this report

Term	Meaning
Workflow	Orchestration code that decides what happens next and performs no input or output itself. It must be deterministic, because it is re-executed on every recovery.
Activity	One step with an external side effect, such as charging a card or calling an API. It may be non-deterministic and runs at least once.
Event	An immutable record appended to the log describing something that has already happened.
History	The ordered events of one workflow run; the complete state of that run.
Replay	Re-executing a workflow against its history so that completed steps resolve from the log.
Shard	One of a fixed number of partitions. A workflow belongs to one shard for its whole life.
Lease	A time-limited claim on a shard, held in a database row and renewed periodically, that makes one node the owner of the shard.
Fencing token	A counter that increases whenever ownership of a shard changes; a write is accepted only with the current value.
Saga	A multi-step transaction that, if it fails part way, is undone by running a compensating action for each step that had succeeded.

Industry has adopted this approach widely. Uber's Cadence and its successor Temporal run very large numbers of workflows for ride and delivery systems [1]; Microsoft's Durable Functions provide the same model for serverless code [3]; and the DBOS project argues that durable execution belongs inside the database rather than in a separate service [4, 5]. The internal mechanisms of such an engine, namely how the log is partitioned, how state is reconstructed deterministically and how ownership of a par-

tition moves safely between machines when one fails, are less often built from first principles. This project builds one, and evaluates it on a cloud cluster under load and under injected failures.

1.2 Motivation

1.2.1 Existing engines depend on several services

The production engines surveyed for this work each require four to six cooperating infrastructure components. Netflix's Conductor uses a database, a search cluster, a queue, a server tier and a worker fleet. Temporal runs four internal services over a database and, for visibility queries, a search cluster [1]. Meta's Shard Manager, which assigns partitions across very large fleets, relies on ZooKeeper for coordination [6, 7]. Each additional component is a separate failure domain that must be configured, monitored and understood during an incident, and for a small deployment the orchestration system can cost more to operate than the application it serves. This project asks whether the same guarantees can be provided with a single dependency.

1.2.2 Partitioning and simplicity are usually traded off

The simple systems are not partitioned and the partitioned systems are not simple. DBOS obtains durable execution with very few moving parts but runs on one node [4]; Temporal and Shard Manager partition work across machines but add a coordination service or a multi-service control plane to decide which machine owns what. Our position is that the two need not be in tension, because a relational database already provides the linearizable compare-and-swap that partition ownership requires [8, 9]. What is needed is a careful use of one row per shard rather than a consensus cluster [10].

1.2.3 Agents defeat byte-level deduplication

Durable execution engines prevent duplicate side effects by recognising a repeated request, and they recognise it by comparing bytes: a hash, an idempotency key or an exact match on recorded arguments. This is sound when the arguments are fixed at the moment a step is first scheduled. It is not sound when the caller is a language-model agent, which after a crash *re-plans* rather than replays: an instruction recorded as "BLR→DEL Friday morning" may be issued again as "Bangalore to Delhi, Friday

AM”. The two strings hash differently and the flight is booked twice. Step repetition of this kind appears as a recurring category in empirical studies of multi-agent failures [11] and in Microsoft’s taxonomy of agentic failure modes [12].

1.3 Problem Statement

The problem addressed in this work is to design and implement a horizontally sharded durable execution engine that runs long, multi-step workflows to completion despite process crashes, node failures and partial failures, using event sourcing, deterministic replay, idempotent activity execution and lease-based ownership; that is deployable on a small cluster with PostgreSQL as its only infrastructure dependency; that detects semantically equivalent repeated actions issued by a recovering agent before the side effect occurs a second time; and to measure its throughput, its sensitivity to its main configuration parameters, and its recovery from injected faults on real cloud infrastructure.

Two limits are part of the statement. The engine targets correctness and fault tolerance at a moderate, measurable scale rather than the throughput of large production deployments. Ownership is obtained from a lease in a linearizable database rather than from a consensus protocol, which makes the database the availability floor of the system.

1.4 Objectives

The work has eight objectives, listed in Table 1.2 with the mechanism that realises each and the chapter in which it is evaluated.

1.5 Scope and Limitations

The scope is the engine, two demonstration applications built on it, and the test bed used to evaluate it. The main limitations are stated here so that the design and the results can be read with them in mind.

1. The engine provides at-least-once execution of activities with strong deduplication, not exactly-once execution. A crash after a side effect has happened but before it has been recorded cannot be told apart from a crash before the side effect. The engine supplies a stable deduplication key that the downstream service can use to close this window.

Table 1.2: Project objectives

No.	Objective	Mechanism	Evaluated
1	Record every state transition as an immutable event so that a run can be reconstructed exactly	Append-only event log	Ch. 6, 7
2	Resume a failed run from its history rather than from the start	Deterministic replay	Ch. 6, 7
3	Never repeat an external side effect across retries	Stable deduplication keys	Ch. 6
4	Partition execution across nodes	Fixed shards, consistent-hash ring	Ch. 7
5	Lose no progress when a shard owner fails	Leases with fencing tokens	Ch. 7
6	Undo multi-step transactions that fail part way	Saga compensation	Ch. 6
7	Detect reworded duplicate agent actions	Journalled semantic verdict	Ch. 6
8	Measure capacity, sensitivity and recovery on cloud hardware	EKS test bed, fault injector	Ch. 7

2. Ownership relies on PostgreSQL as the single source of truth. The cloud evaluation used one database host with no standby, so it says nothing about database high availability.

3. The event log is never garbage collected; retention is future work.

4. Semantic deduplication is opt-in and conservative. A false negative is no worse than exact matching, but a false positive would suppress an action the user wanted, so the semantic mode must be requested per activity and every verdict is journalled.

5. All cloud measurements come from one availability zone on burstable (“flex”) instance types, over individual runs of a few minutes. They establish the shape of the behaviour on this configuration, not absolute capacity for other hardware.

1.6 Contributions

The contributions of this work are as follows.

1. A durable execution engine in which event sourcing, replay, a durable task queue, timers, signals, sagas and shard failover are implemented over PostgreSQL alone, with no message broker or coordination service.

2. Fencing at the persistence layer: every transaction that writes shard state re-reads the shard's fencing token with `SELECT ... FOR UPDATE` and commits only if the token is still the writer's. On the cloud test bed this rejected every write attempted by a stale owner after a process freeze.
3. A two-level mapping in which workflows hash to a fixed set of shards and shards are assigned to nodes by a consistent-hash ring, so that a change of membership moves ownership but no data.
4. A semantic idempotency check that compares the meaning of an agent's intended action with the actions already in the run's history and journals the verdict, keeping replay deterministic and auditable.
5. A cloud evaluation on Amazon EKS covering idle overhead, latency floor, closed- and open-loop capacity, sensitivity to workers, shards and replicas, and recovery from six classes of injected fault, together with the evaluation harness and its validity checks.

1.7 Organisation of the Report

Chapter 2 reviews related work on durable execution, partition management, ownership and fencing, sagas, and semantic similarity, and identifies the gap this work addresses. Chapter 3 states the requirements and the analysis that led to the chosen approach. Chapter 4 presents the high-level and low-level design. Chapter 5 describes the implementation and Chapter 6 the testing methodology, including the cloud test bed and the fault-injection procedure. Chapter 7 reports and discusses the results, and Chapter 8 concludes and outlines future work.

CHAPTER - 2

LITERATURE SURVEY

This chapter reviews the work on which the engine is built. Six bodies of literature are relevant: durable execution and stateful serverless systems; the partitioning of work across machines; leases, coordination and fencing; long-running transactions and sagas; the failure modes of agent systems built on language models; and semantic similarity and caching. Each section closes with the point taken from it into the design. The chapter ends with a comparison of the closest systems and a statement of the gap.

2.1 Durable Execution and Stateful Serverless Workflows

The principle that a computation can be recovered by re-executing it against a log of its own decisions is older than the term durable execution. Lamport showed that a deterministic state machine fed the same ordered inputs reaches the same state [13], and the write-ahead log of classical transaction processing applies the same reasoning to database pages [14]. Log-centred systems such as Kafka later made an ordered, durable log the main interface between services [15]. Durable execution engines apply the idea at the level of application code: the inputs to the state machine are the results of the workflow's external calls, and the log is the workflow's event history.

Temporal, which grew out of Uber's Cadence, is the most widely used engine of this kind [1]. Workflow code is written in an ordinary programming language under a determinism contract; each external call is recorded as a pair of scheduled and completed events, and recovery replays the code against the history. Temporal partitions histories into a fixed number of shards, and a membership service assigns shards to history service hosts. The architecture is proven at large scale, but a deployment comprises frontend, history, matching and worker services over a database, with an optional search cluster for visibility queries. Restate takes a similar programming model but embeds its own log and storage in a single binary [2].

Durable Functions brought the model to serverless platforms. Burckhardt *et al.* give a formal semantics for orchestrations, entities and replay, and prove that replay is correct

when the orchestration code is deterministic [3]. Netherite replaced the original storage provider with partitioned, speculatively committed logs and reported large gains in throughput and latency [19]. Ambrosia earlier proposed *virtual resiliency*, in which logging and deterministic replay make a distributed application behave as if it never failed, at the cost of requiring every component to be deterministic [16].

Two research systems from the serverless community address the same problem with a shared log. Beldi provides exactly-once semantics and transactions for stateful serverless workflows by logging every read and write in a linked data-access log kept in a key-value store [17]. Boki builds a shared log into the function runtime and offers durable workflows, object storage and message queues as libraries over it [18]. Both achieve strong guarantees, and both depend on a purpose-built logging layer.

DBOS argues the opposite direction: that the state of an operating system, including the state of running programs, belongs in a database [4]. Its durable workflows checkpoint each step into PostgreSQL, so that recovery needs nothing other than the database. R3 extends the approach to record and replay of database-backed applications, so that a past execution can be reproduced and modified retroactively [5]. DBOS is the system closest in spirit to this project. It does not, however, partition workflows across nodes with explicit ownership; recovery is driven by the database and the executor that originally ran the workflow.

Taken into the design. The workflow and activity split, the determinism contract and the recovery-by-replay model come from Temporal and Durable Functions. The decision to use PostgreSQL as the only store follows DBOS. The engine differs from all of them in combining a single-dependency deployment with explicit, fenced shard ownership.

2.2 Partitioning Work Across Machines

A system that runs on more than one machine must decide which machine is responsible for each piece of state. Consistent hashing, introduced by Karger *et al.* for web caches [20], maps keys and machines onto the same circle so that adding or removing a machine moves only the keys adjacent to it. Dynamo applied the technique to a

replicated key-value store and introduced virtual nodes, several positions per physical machine, to even out the load [21].

Hashing keys directly to machines has a drawback when the state behind each key is large or when ownership must be transferred carefully: every membership change touches many independent keys. Systems with expensive ownership transfer therefore introduce a fixed intermediate layer. Slicer, at Google, maps keys to slices and assigns slices to tasks, rebalancing the assignment from observed load [22]. Shard Manager at Meta generalises this into a platform that places and migrates shards for hundreds of applications, and reports that a fixed shard count with movable assignment is the dominant pattern in practice [6]. Borg schedules the processes that host such shards [33].

Taken into the design. The engine uses the two-level scheme of Slicer and Shard Manager: a workflow hashes to one of a fixed number of shards for its whole life, and shards are assigned to nodes by a consistent-hash ring with virtual nodes. A change of membership moves ownership but no data, because all data is in the shared database.

2.3 Leases, Coordination and Fencing

Assigning a shard to a node is only half of the problem; the system must also ensure that a node that has lost a shard stops writing to it. Gray and Cheriton introduced leases, time-bounded grants of authority that expire unless renewed, so that a crashed holder cannot block others forever [8]. A lease is safe only under assumptions about clock drift and process pauses. A holder that is paused, for example by garbage collection or by a virtual machine being descheduled, may resume after its lease has expired and act on authority it no longer has. Kleppmann describes this failure and the standard remedy: a *fencing token*, a number that increases with every grant, which the storage layer checks on every write [9].

Production systems usually obtain leases from a coordination service. Chubby provides coarse-grained locks with sequencers that serve as fencing tokens [23], and ZooKeeper provides the primitives from which leader election and locks are built [7]. Both rest on a consensus protocol such as Paxos or Raft [10]. The impossibility result of Fischer, Lynch and Paterson shows why some timing assumption is unavoidable: in a purely

asynchronous system no protocol can both tolerate a crash and always terminate [24]. Leases embody that assumption explicitly.

Taken into the design. A coordination service is not required if the store that holds the protected data can itself perform a linearizable compare-and-swap. PostgreSQL can, through row locks. The engine therefore keeps one lease row per shard and makes every write transaction check the fencing token under `SELECT ... FOR UPDATE` [38]. The token check happens in the same transaction as the write, which is the property the literature identifies as essential.

2.4 Long-Running Transactions and Sagas

Garcia-Molina and Salem defined a saga as a long-lived transaction written as a sequence of shorter transactions, each paired with a compensating transaction that semantically undoes it [25]. If the saga fails part way, the compensations for the completed steps are run in reverse order. Helland later argued that applications which span services cannot rely on distributed transactions at all, and must be built from idempotent operations on individually addressed entities [26]. Recent work on microservices has examined the isolation anomalies that sagas permit and proposed remedies such as semantic locks and versioning [27].

Two points from this literature matter here. A compensation is an action with its own side effects and can itself fail, so it must be retried and deduplicated in the same way as a forward step. The order and completeness of compensation must also survive a crash in the middle of rolling back.

Taken into the design. Compensations are ordinary activities, registered as each forward step completes and run in reverse order. Because they are activities, their scheduling and completion are recorded in the event history, and a crash during rollback is recovered by replay like any other.

2.5 Failure Modes of Agent Systems

Agents built on language models are a new class of long-running workflow. An agent plans, calls tools that act on the world, observes the results and plans again. Cemri

et al. analysed traces from several multi-agent frameworks and organised the observed failures into a taxonomy; step repetition and failure to recognise that a task was already complete appear among the recurring categories [11]. Microsoft's AI Red Team reached a similar list from the security side, including repeated or unintended actions after an interruption [12].

The relevance to durable execution is specific. A conventional engine records the arguments of each step when it is scheduled, so an engine-level retry of that step uses identical arguments. An agent that crashes between two planning steps, however, will re-run the second planning step on recovery, and at a non-zero sampling temperature it may describe the same action in different words. The engine sees a new step with new arguments.

2.6 Semantic Similarity and Caching

Sentence-BERT made it practical to compare the meaning of two sentences by the cosine similarity of fixed-size embeddings [28], and MiniLM showed that a small distilled model can produce such embeddings at a fraction of the cost of the original [29]. Semantic caches for language-model services build on this. GPTCache returns a stored answer when a new prompt is close enough to a cached one [30]; MeanCache adapts the threshold to each user [31]; and vCache argues that a single global threshold is unreliable and learns per-entry thresholds with an error bound [32].

A semantic cache and a semantic deduplication check fail in opposite directions. A false hit in a cache returns a slightly wrong answer, which the user can retry. A false match in a deduplication check suppresses an action the user wanted, which is worse than missing a duplicate. The cache literature therefore informs the mechanism but not the policy.

Taken into the design. The engine uses a small sentence-embedding model and a cosine threshold, but adopts a conservative policy: semantic matching is opt-in per activity, candidates are restricted to completed activities of the same type in the same run, an uncertain band between two thresholds is escalated rather than decided, and every verdict is recorded in the history.

2.7 Comparison of Related Systems

Table 2.1 compares the systems closest to this work on the properties that the engine is designed to combine.

Table 2.1: Comparison of related systems

System	Infrastructure	Partitioned	Ownership safety	Semantic dedup
Temporal [1]	4 services + DB (+ search)	Yes	Range IDs per shard	No
Durable Functions [3, 19]	Cloud storage or Netherite	Yes	Partition leases	No
Beldi [17]	Serverless + key-value store	By key	Logged conditional writes	No
Boki [18]	Custom shared log	Yes	Log sequencer	No
DBOS [4]	PostgreSQL	No	Database transactions	No
Shard Manager [6]	ZooKeeper + control plane	Yes	Coordination service	Not applicable
GPTCache [30]	Vector store	Not applicable	Not applicable	Cache only
This work	**PostgreSQL**	**Yes**	**Fencing token in each write**	**Journalled verdict**

2.8 Research Gap

Three observations follow from the survey. First, the engines that partition workflows across machines do so with several cooperating services or a coordination service, while the engine that needs only a database does not partition. No system in the survey provides partitioned, fenced ownership using only the database that already stores the workflow state. Second, fencing is well understood in principle, but published descriptions rarely report how often a stale owner actually attempts a write in a real deployment, or how long ownership takes to move after a failure. Third, every engine deduplicates steps by exact comparison of arguments, which is sound for engine retries but not for an agent that re-plans after a crash, and the semantic caching work that could detect such repeats is designed for a setting in which a false match is cheap.

This project addresses the three points together: an engine with partitioned, fenced ownership over PostgreSQL alone; a cloud evaluation that measures ownership transfer and stale writes under injected faults; and a semantic deduplication check whose verdict is recorded in the history so that replay remains deterministic.

CHAPTER - 3

REQUIREMENTS AND ANALYSIS

This chapter states what the engine must do and the conditions under which it must do it. Functional requirements are numbered FR-1 to FR-12 and non-functional requirements NFR-1 to NFR-8, so that later chapters can refer to them. The chapter ends with the analysis that led to the chosen approach.

3.1 Functional Requirements

Table 3.1 lists the functional requirements. They follow from the objectives in Table 1.2.

3.2 Non-Functional Requirements

3.3 Software and Hardware Requirements

The engine is written in Go 1.25 and uses the `pgx` driver for PostgreSQL. PostgreSQL 16 with the `pgvector` extension stores all state, including the 384-dimensional intent embeddings used for semantic deduplication. Embeddings are produced by an `all-MiniLM-L6-v2` model served over HTTP. Metrics are exported in Prometheus format. Development and local testing used Docker Compose; the cloud evaluation used Amazon EKS with Terraform for provisioning. Table 3.3 gives the hardware used in each setting.

3.4 Assumptions and Constraints

The design rests on the following assumptions.

1. PostgreSQL provides serializable row locks and durable commits. The engine does not defend against a database that loses committed data.
2. Clocks on engine nodes may drift by a small amount relative to the lease period. Lease expiry is computed with the database clock, so drift between engine nodes does not affect safety; fencing protects against pauses longer than the lease.

Table 3.1: Functional requirements

ID	Requirement
FR-1	A client can start a workflow of a registered type with an identifier and an input. Starting the same identifier twice returns the existing run rather than creating a second one.
FR-2	Every state change of a run is appended to its history as an immutable event with a gap-free sequence number.
FR-3	A run interrupted by a crash at any point resumes from its history and reaches the same result it would have reached without the crash.
FR-4	Activities are executed by workers through a durable task queue, with a configurable retry policy and per-task lock timeout.
FR-5	Each activity receives a deduplication key that is identical on every attempt and every replay.
FR-6	Workflows can wait on durable timers and on named signals sent by clients; both survive restarts.
FR-7	A workflow can register compensations; on failure they run in reverse order of registration, and rollback survives a crash.
FR-8	Workflows are partitioned into a fixed number of shards, each owned by at most one node at a time.
FR-9	When a node fails, its shards are claimed by surviving nodes and their runs continue without loss of progress.
FR-10	A write by a node that no longer owns the shard is rejected.
FR-11	An activity may opt in to semantic deduplication. A request whose meaning matches a completed activity of the same type in the same run returns that activity's recorded result instead of executing again.
FR-12	Clients can query the status and history of a run, and operators can read metrics for throughput, latency, ownership and errors.

Table 3.2: Non-functional requirements

ID	Property	Requirement
NFR-1	Correctness	No completed activity is executed again by the engine because of a crash, retry or ownership change, apart from the unavoidable window described in Section 4.2.4.
NFR-2	Recovery	After a node is killed, its shards are claimed within one lease period plus one renewal interval.
NFR-3	Dependencies	PostgreSQL is the only infrastructure dependency. The embedding service is needed only by activities that use semantic deduplication, and never during replay.
NFR-4	Scalability	Throughput increases when nodes or workers are added, until the database becomes the limit.
NFR-5	Efficiency	An idle cluster places a small, bounded load on the database.
NFR-6	Observability	Every node exports Prometheus metrics and structured logs; every semantic verdict is recorded with its inputs.
NFR-7	Portability	The engine runs as a single static binary in a container on Kubernetes or directly on a host.
NFR-8	Testability	Fault injection hooks allow crashes, pauses and database outages to be triggered at named points in tests.

Table 3.3: Hardware used for development and evaluation

Role	Instance	Resources
Development	Laptop / cloud container	4–8 vCPU, 16 GB, Docker
Engine nodes (cloud)	3 × m7i-flex.large	2 vCPU, 8 GB each
Load harness (cloud)	1 × c7i-flex.large	2 vCPU, 4 GB
Database (cloud)	1 × m7i-flex.large	2 vCPU, 8 GB, gp3 volume

3. Workflow code is deterministic. The engine provides deterministic replacements for time, random numbers and unique identifiers, and detects some violations at replay time, but cannot prove determinism in general.
4. External services that are called by activities accept an idempotency key, or their operations are naturally idempotent.

Two constraints were fixed at the start. The engine must not introduce any infrastructure dependency beyond PostgreSQL, and all cloud resources must fit the free-tier and low-cost instance types available to the project, which ruled out larger database hosts and multi-zone deployments.

3.5 Analysis of the Chosen Approach

Three decisions shape the rest of the design, and each was taken by comparing alternatives against the requirements.

State: event history or checkpoints. A checkpointing design stores the current state of each run and overwrites it after each step. It is simple, but it records what the state is and not how it was reached, so a crash between a side effect and the checkpoint leaves no trace of the side effect, and debugging requires extra logging. An event history records every decision; the current state is a function of the history, and the history is also the audit trail. The cost is replay time for long histories. Event sourcing was chosen because FR-2, FR-3 and FR-11 all depend on the history being the single source of truth.

Ownership: coordination service or database rows. A coordination service such as ZooKeeper or etcd provides leases and watches, but violates NFR-3 and adds a second system whose view of ownership can disagree with the database. More importantly, a lease held in one system does not by itself prevent a stale write to another: the token must be checked where the data is written. Keeping the lease and the fencing token in the same database as the workflow state lets a single transaction check ownership and write the data, which satisfies FR-10 directly.

Task delivery: message broker or database queue. A broker would offer higher raw throughput, but an activity's completion must be recorded in the history atomically with the removal of its task; with a separate broker this requires an outbox and a relay. A queue table polled with `SELECT ... FOR UPDATE SKIP LOCKED` gives atomic hand-off within one transaction. The cost, measured in Chapter 7, is polling load on the database when the cluster is idle and lock contention at high load.

The price of these decisions is that the database is both the availability floor and the throughput ceiling of the system. The evaluation in Chapter 7 was designed to locate that ceiling and to show what happens beyond it.

CHAPTER - 4

SYSTEM DESIGN

This chapter describes the design of the engine in two parts. The high-level design (Section 4.1) presents the architecture, the separation of workflow and activity code, the mapping of workflows to shards and nodes, and the ownership protocol. The low-level design (Section 4.2) gives the data model, the workflow task loop, the crash-window analysis, sagas and the semantic deduplication path.

4.1 High-Level Design

4.1.1 Governing principle

One rule governs the design: *the event history is the only state of a workflow run, and every decision is committed to it before it is acted on*. A node holds caches, worker goroutines and open connections, but nothing that would be lost if the process were killed. Recovery therefore never needs to reconstruct a node's memory; it needs only to replay a run's history, on any node that currently owns the run's shard.

4.1.2 Architecture

Fig. 4.1 shows the components. Clients reach any engine node through an HTTP API to start runs, send signals and read status. Each node is one Go process with four parts: the API server, a lease manager that acquires and renews shard leases, a pool of workflow workers that execute workflow code, and a pool of activity workers that execute side effects. All nodes run the same binary; any of them can own any shard. PostgreSQL holds the event log, the task queue, timers, leases and intent embeddings. There is no other shared component.

4.1.3 Separation of workflow and activity

Workflow code and activity code have opposite requirements, and the engine treats them differently. Workflow code decides what to do next. It is re-executed on ev-

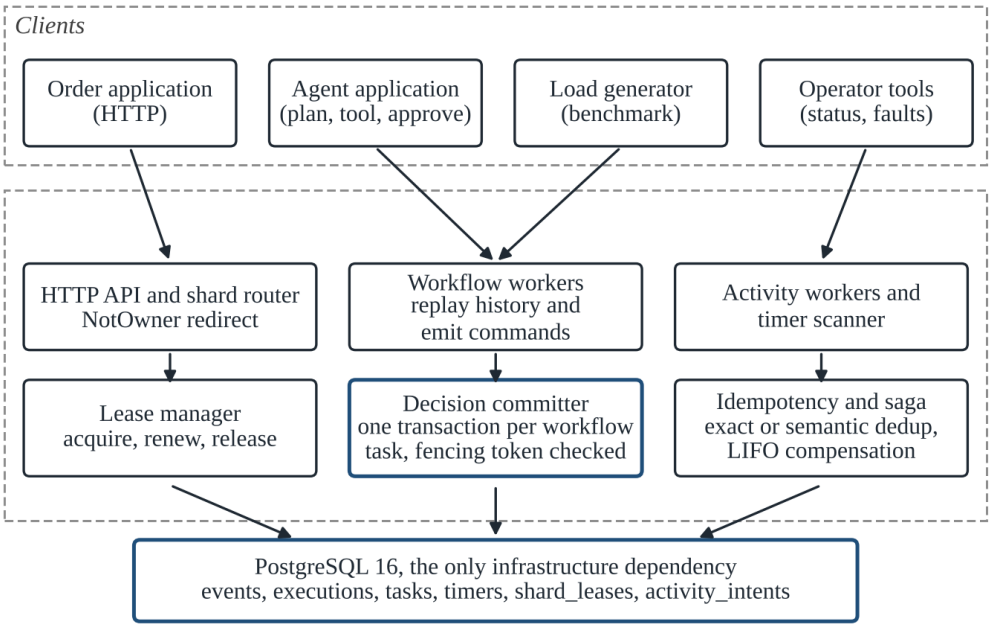


Fig. 4.1: System architecture. The engine node is one Go process, replicated N times; PostgreSQL is the only shared component.

ery recovery, so it must be deterministic and must perform no input or output; it calls `ExecuteActivity`, `Sleep`, `WaitForSignal` and `SideEffect` on a context object, each of which either returns a result recorded in the history or suspends the workflow until one is available. Activity code performs the side effects. It may be non-deterministic, may fail and may take a long time; it runs at least once and its result is recorded when it completes.

A workflow function in the engine's Go SDK has the following shape.

Listing 4.1: A workflow with two activities and a saga compensation

```
func Order(ctx workflow.Context, in OrderInput) (Receipt, error) {
    s := saga.New(in.OrderID, runner(ctx))
    var pay Payment
    if err := ctx.ExecuteActivity("charge_card", in.Card).Get(&pay); err !=
        nil {
        return Receipt{}, err
    }
    s.AddCompensation("refund_card", pay.ID)   // only after success
    var res Reservation
```

```
9       if err := ctx.ExecuteActivity("reserve_stock", in.Items).Get(&res); err
            != nil {
10          return Receipt{}, s.Compensate()        // runs refund_card
11      }
12      return Receipt{Payment: pay.ID, Reservation: res.ID}, nil
13  }
```

4.1.4 Two-level mapping of workflows to nodes

A workflow identifier is hashed to one of S shards, with $S = 16$ by default, and the shard never changes for the life of the run. Shards are assigned to nodes by a consistent-hash ring with 150 virtual nodes per engine node (Fig. 4.2). The ring expresses a preference only: a node reaches for the shards the ring assigns to it, and holding a shard is decided by its lease. Because all data lives in the shared database, moving a shard between nodes moves no data; the new owner simply begins to poll that shard's task rows.

Two rules make the preference converge after failures. A node may adopt a shard that the ring does not assign to it if the shard's lease has been expired for more than twice the lease period, which is taken as evidence that the owner is not coming back. A node that holds a shard the ring assigns to another node returns it as soon as that node answers its health check. The first rule prevents a dead node's shards from stalling forever; the second keeps the load even after the node recovers. The delay in the first rule avoids needless handovers after a short pause or a rolling restart, and its cost appears in the freeze results of Chapter 7.

4.1.5 Leases and the fencing invariant

Each shard has one row in `shard_leases` holding the owner, an expiry time and a fencing token. A node acquires a free or expired lease with a single conditional update that increments the token and sets the expiry to the database's `now()` plus the lease period of 10 s. It renews every 3 s, so a node has three attempts before its lease lapses. Expiry is computed on the database clock, so clock skew between engine nodes does not matter.

A lease alone is not safe. A node that is paused for longer than the lease period resumes believing it still owns its shards, while another node has already taken them over. The engine therefore enforces the following invariant at the point of writing:

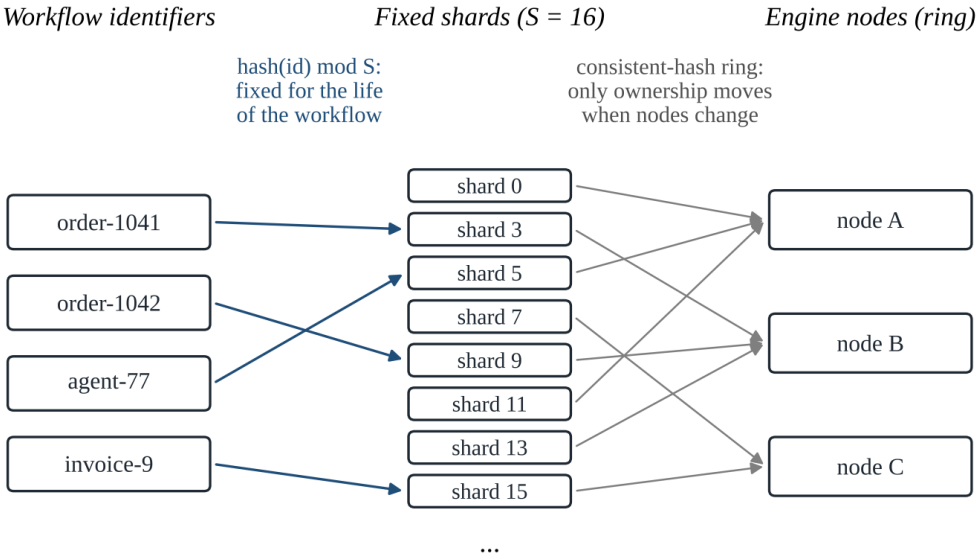


Fig. 4.2: Two-level mapping. Workflows hash to a fixed set of shards; shards are placed on nodes by a consistent-hash ring.

> *Every transaction that changes the state of a shard first reads that shard's fencing token with* `SELECT ... FOR UPDATE`*, and commits only if the token equals the one the writer was granted.*

The row lock is held until the transaction ends, so ownership cannot change between the check and the commit; a competing acquire waits for the lock and then increments the token. A stale owner's transaction finds a higher token and rolls back, and none of its writes become visible. Fig. 4.3 shows the sequence as it occurred on the cloud test bed when an engine process was frozen for 35 s.

The token check is also the reason a separate coordination service is not needed. A lease obtained from ZooKeeper or etcd would still require a token check where the data is written; with the lease in the same database, the check and the write are one transaction.

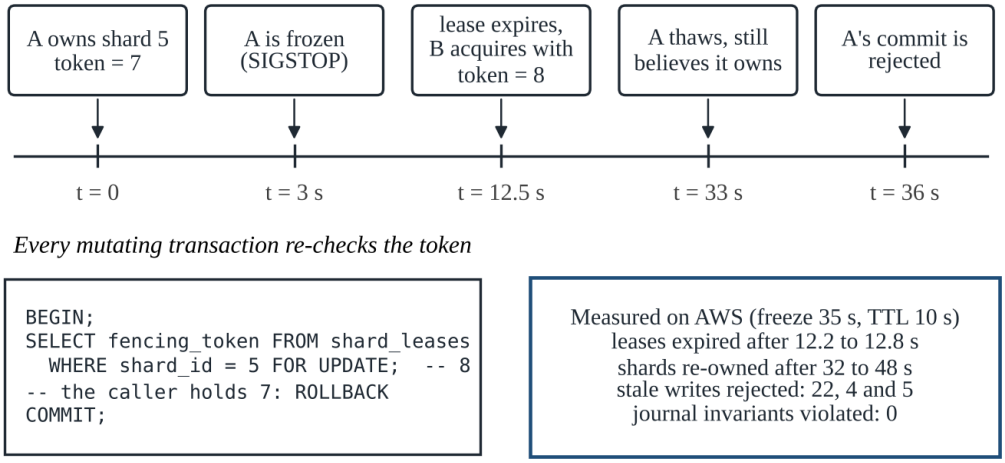


Fig. 4.3: Lease expiry and fencing of a stale owner, with the values measured in the freeze trials.

4.2 Low-Level Design

4.2.1 Data model

Table 4.1 lists the six tables. Every table except `shard_leases` carries the shard identifier as the first column of its primary key or its main index, so that each node's queries touch only the rows of the shards it owns.

Table 4.1: Persistent data model

Table	Key	Contents
`events`	(shard, workflow, run, seq)	Event type and JSON payload; append-only, gap-free `seq`
`executions`	(shard, workflow)	Run identifier, type, status and the last sequence number
`tasks`	(shard, id)	Workflow and activity tasks with visibility time, lock expiry, lock token and attempt count
`timers`	(shard, workflow, run, seq)	Fire time of each durable timer
`shard_leases`	shard	Owner, fencing token and expiry
`activity_intents`	(shard, workflow, run, seq)	Canonical intent text and its 384-dimensional embedding

The primary key of `events` is a second line of defence: even if two writers passed the fencing check, they could not both append the same sequence number.

4.2.2 The workflow task loop

A workflow makes progress only when a workflow task for it is processed. Such a task is created when the run starts and whenever something the run is waiting for happens: an activity completes or fails, a timer fires or a signal arrives. Fig. 4.4 shows how a worker processes one task.

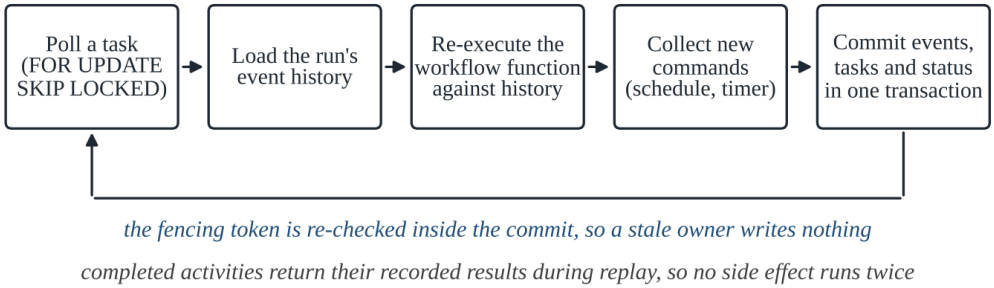


Fig. 4.4: The workflow task loop. All writes of one decision commit in a single fenced transaction.

The worker loads the run's history and executes the workflow function from the beginning. Each call on the workflow context is numbered by its position in program order, $k = 0, 1, 2, \ldots$. If the history holds a result for call k, the call returns that result immediately. The first call with no recorded result ends the replay: the worker collects the new commands the workflow has issued (schedule an activity, start a timer, complete the run) and commits them as one *decision*. The commit performs, in order: the fencing check; admission of the run for a start, using the `executions` primary key as the start-idempotency check; appending the new events; inserting the new tasks, timers and intents; updating the execution status; and deleting the processed task. Either all of these become visible or none do.

Using the program-order index k as the join key between code and history is what makes replay exact. A workflow that issues the same calls in the same order matches its history call by call. A workflow whose code has been changed in a way that alters the order is detected on replay, because the command at position k no longer matches the recorded event type, and the run is failed with a non-determinism error instead of continuing with a corrupted history.

Workflow code is run on a cooperative scheduler. Each workflow coroutine runs until it blocks on a future, and the scheduler then runs the next ready one, so that a workflow that starts several activities in parallel is still executed in a deterministic order. Time, random numbers and unique identifiers are obtained from the context, which returns recorded values during replay.

4.2.3 Task queue, timers and signals

Workers poll the `tasks` table with `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1`, restricted to the shards the node owns and to rows whose visibility time has passed and whose lock has expired. The selected row is locked by setting `locked_until` and a new `lock_token`. `SKIP LOCKED` lets many workers poll the same shard without waiting on each other. A long-running activity extends its lock with a heartbeat; if the worker dies, the lock expires after 30 s and the task is delivered again.

Timers are rows with a fire time. A timer loop on the owning node selects due timers, appends a `TimerFired` event and creates a workflow task, in one fenced transaction. A signal is appended directly to the target run's history as a `SignalReceived` event together with a workflow task, so a signal sent while the run's shard is moving between nodes is not lost.

4.2.4 Retries and the crash-window analysis

When an activity fails, the workflow worker applies the activity's retry policy: an initial interval of 1 s, a backoff coefficient of 2, a maximum interval of 60 s and at most five attempts by default, with a list of error types that are not retried. The retry is a new task with a later visibility time; the arguments are those recorded when the activity was first scheduled.

Table 4.2 considers each point at which a crash can occur during one activity. In three of the four windows recovery is safe without help from the activity. In window C the external effect has happened but the engine has no record of it, and the activity will be called again.

Window C cannot be closed by the engine, because the external service and the engine's database are separate systems with no common transaction. The engine provides the means for the external service to close it: every activity receives a deduplication key

Table 4.2: Crash windows for one activity execution

	Crash point	Recovery	Safe
A	Activity scheduled, not yet polled	Another worker polls the task; the activity runs once	Yes
B	Task polled, external call not made	The lock expires and the task is redelivered; the activity runs once	Yes
C	External call succeeded, completion not recorded	The lock expires and the task is redelivered; the external call is made again	No
D	Completion recorded, workflow task not yet processed	Replay resolves the activity from history; it is not run again	Yes

formed from the run identifier and the program-order index k, which is the same on every attempt and every replay. An activity that passes this key as the idempotency key of the external call turns a repeated call into a no-op at the service. The engine's guarantee is therefore at-least-once execution of activities and exactly-once recording of their results.

4.2.5 Saga compensation

Compensations are managed by a small saga object created inside the workflow (Listing 4.1). A compensation is added with `AddCompensation` only after its forward step has returned successfully, so a step that failed or never ran has nothing to undo. When a later step fails, `Compensate` runs the registered compensations as activities in reverse order of registration.

The stack of compensations is not written to the history. It is a pure function of the history, because on replay the workflow follows the same path, observes the same successes and registers the same compensations in the same order. Recording it would add an event per step and would create a second copy that could disagree with the first. Each compensation is scheduled and completed through the history like any other activity, with its own retries and deduplication key, so a crash in the middle of a rollback resumes at the first compensation that has not completed.

4.2.6 Semantic deduplication

The source of reworded duplicates is narrower than it may appear. An engine retry of a scheduled activity reuses the recorded arguments, so exact matching is enough. Reworded duplicates arise when an agent re-plans: replay resolves an earlier plan and its tool call from history, but a later planning step whose result was not yet recorded runs again and may express the same intent in different words.

The similarity check is itself non-deterministic, since it depends on an embedding model and on a threshold. If it were evaluated in the activity worker, it would be re-evaluated on replay and could give a different answer, and the history would diverge. The engine therefore evaluates it once, on the workflow path, and records the verdict as an event, in the same way as a `SideEffect`. Listing 4.2 gives the procedure.

Listing 4.2: Semantic deduplication of an activity at position k

```
first execution:
  intent     := canonical(type, normalised and sorted arguments)
  candidates := completed semantic activities of the same type in this run
  v          := embed(intent)                      // 384 dimensions
  (j, s)     := argmax over candidates of cosine(v, v_j)
  verdict    := DUPLICATE if s >= tau_high (0.90)
                DISTINCT  if s <  tau_low  (0.85)
                ESCALATE  otherwise
  append SemanticDedupEvaluated{k, verdict, j, s, thresholds, model}
  DUPLICATE: return the recorded result of activity j; schedule nothing
  DISTINCT:  schedule the activity as usual
  ESCALATE:  wait for an approval signal, or execute (per configuration)
replay:
  read SemanticDedupEvaluated{k} and apply the recorded verdict
  (no embedding call, no similarity computation)
```

Three properties follow. Replay needs no embedding service, so recovery does not depend on its availability. Every verdict is stored with its similarity, thresholds, matched activity and model name, so a suppressed action can be explained after the fact. The candidate set is restricted to completed activities of the same type in the same run, which keeps the comparison small and makes a false match across unrelated runs impossible. The intent embeddings are stored in `activity_intents` with the `pgvector` type.

4.2.7 Node runtime

At start-up a node applies the schema migrations, connects to PostgreSQL, builds the ring from the configured list of cluster nodes and starts the lease manager. For each shard it acquires, it starts workflow, activity and timer loops for that shard; when it loses a shard it stops them at once. The fencing check would reject their writes in any case, but stopping them avoids polling load on a shard the node no longer owns. On a graceful shutdown the node stops taking new tasks, waits for in-flight tasks within the shutdown grace period and releases its leases, so that the next owner can take over without waiting for expiry. The defaults are four workflow workers and eight activity workers per node; Chapter 7 measures the effect of changing them.

REFERENCES

[1] Temporal Technologies, "Temporal platform documentation," [Online]. Available: `https://docs.temporal.io/`. Accessed: Oct. 2026.

[2] Restate, "What is durable execution?" [Online]. Available: `https://restate.dev/what-is-durable-execution/`. Accessed: Oct. 2026.

[3] S. Burckhardt, C. Gillum, D. Justo, K. Kallas, C. McMahon and C. S. Meiklejohn, "Durable functions: Semantics for stateful serverless," *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, Art. 133, 2021.

[4] A. Skiadopoulos, Q. Li, P. Kraft, K. Kaffes, D. Hong, S. Mathew, D. Bestor, M. Cafarella, V. Gadepally, G. Graefe, J. Kepner, C. Kozyrakis, T. Kraska, M. Stonebraker, L. Suresh and M. Zaharia, "DBOS: A DBMS-oriented operating system," *Proc. VLDB Endow.*, vol. 15, no. 1, pp. 21–30, 2022.

[5] Q. Li, P. Kraft, M. Cafarella, C. Demiralp, G. Graefe, C. Kozyrakis, M. Stonebraker, L. Suresh, X. Yu and M. Zaharia, "R3: Record-replay-retroaction for database-backed applications," *Proc. VLDB Endow.*, vol. 16, no. 11, pp. 3085–3097, 2023.

[6] S. Lee, Z. Guo, O. Sunercan, J. Ying, T. Kooburat, S. Biswal, J. Chen, K. Huang, Y. Cheung, Y. Zhou, K. Veeraraghavan, B. Damani, P. M. Ruiz, V. Mehta and C. Tang, "Shard Manager: A generic shard management framework for geo-distributed applications," in *Proc. 28th ACM Symp. Operating Systems Principles (SOSP)*, 2021, pp. 553–569.

[7] P. Hunt, M. Konar, F. P. Junqueira and B. Reed, "ZooKeeper: Wait-free coordination for internet-scale systems," in *Proc. USENIX Annual Technical Conf.*, 2010, pp. 145–158.

[8] C. Gray and D. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in *Proc. 12th ACM Symp. Operating Systems Principles (SOSP)*, 1989, pp. 202–210.

[9] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.

[10] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conf.*, 2014, pp. 305–319.

[11] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez and I. Stoica, "Why do multi-agent LLM systems fail?" arXiv:2503.13657, 2025.

[12] R. S. S. Kumar, P. Bryan, C. Seifert, N. Chawla, A. Kumar and Y. Zunger, "Taxonomy of failure modes in agentic AI systems," Microsoft AI Red Team, White paper, Apr. 2025.

[13] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[14] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA: Morgan Kaufmann, 1992.

[15] J. Kreps, N. Narkhede and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, 2011.

[16] J. Goldstein, A. Abdelhamid, M. Barnett, S. Burckhardt, B. Chandramouli, D. Gehring, N. Lebeck, C. Meiklejohn, U. F. Minhas, R. Newton, R. A. Peshawaria, T. Zaccai and I. Zhang, "A|MBROSIA: Providing performant virtual resiliency for distributed applications," *Proc. VLDB Endow.*, vol. 13, no. 5, pp. 588–601, 2020.

[17] H. Zhang, A. Cardoza, P. B. Chen, S. Angel and V. Liu, "Fault-tolerant and transactional stateful serverless workflows," in *Proc. 14th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2020, pp. 1187–1204.

[18] Z. Jia and E. Witchel, "Boki: Stateful serverless computing with shared logs," in *Proc. 28th ACM Symp. Operating Systems Principles (SOSP)*, 2021, pp. 691–707.

[19] S. Burckhardt, B. Chandramouli, C. Gillum, D. Justo, K. Kallas, C. McMahon, C. S. Meiklejohn and X. Zhu, "Netherite: Efficient execution of serverless workflows," *Proc. VLDB Endow.*, vol. 15, no. 8, pp. 1591–1604, 2022.

[20] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine and D. Lewin, "Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the World Wide Web," in *Proc. 29th ACM Symp. Theory of Computing (STOC)*, 1997, pp. 654–663.

[21] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM Symp. Operating Systems Principles (SOSP)*, 2007, pp. 205–220.

[22] A. Adya, D. Myers, J. Howell, J. Elson, C. Meek, V. Khemani, S. Fulger, P. Gu, L. Bhuvanagiri, J. Hunter, R. Peon, L. Kai, A. Shraer, A. Merchant and K. Lev-Ari, "Slicer: Auto-sharding for datacenter applications," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2016, pp. 739–753.

[23] M. Burrows, "The Chubby lock service for loosely-coupled distributed systems," in *Proc. 7th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2006, pp. 335–350.

[24] M. J. Fischer, N. A. Lynch and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *J. ACM*, vol. 32, no. 2, pp. 374–382, 1985.

[25] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1987, pp. 249–259.

[26] P. Helland, "Life beyond distributed transactions: An apostate's opinion," in *Proc. 3rd Conf. Innovative Data Systems Research (CIDR)*, 2007, pp. 132–141.

[27] E. Y. Daraghmi, C.-P. Zhang and S.-M. Yuan, "Enhancing saga pattern for distributed transactions within a microservices architecture," *Applied Sciences*, vol. 12, no. 12, Art. 6242, 2022.

[28] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[29] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 5776–5788.

[30] F. Bang, "GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings," in *Proc. 3rd Workshop for NLP Open Source Software (NLP-OSS)*, 2023, pp. 212–218.

[31] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar and M. A. Gulzar, "MeanCache: User-centric semantic caching for LLM web services," in *Proc. IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*, 2025.

[32] L. G. Schroeder, S. Liu, A. Cuadron, M. Zhao, S. Krusche, A. Kemper, M. Zaharia and J. E. Gonzalez, "vCache: Verified semantic prompt caching," arXiv:2502.03771, 2025.

[33] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. 10th European Conf. Computer Systems (EuroSys)*, 2015, Art. 18.

[34] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[35] B. Schroeder, A. Wierman and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proc. 3rd USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2006, pp. 239–252.

[36] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[37] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds and C. Rosenthal, "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.

[38] The PostgreSQL Global Development Group, "PostgreSQL 16 documentation," [Online]. Available: `https://www.postgresql.org/docs/16/`. Accessed: Oct. 2026.